

Linguagens formais, autômatos e paradigmas de programação

Disciplina: Compiladores

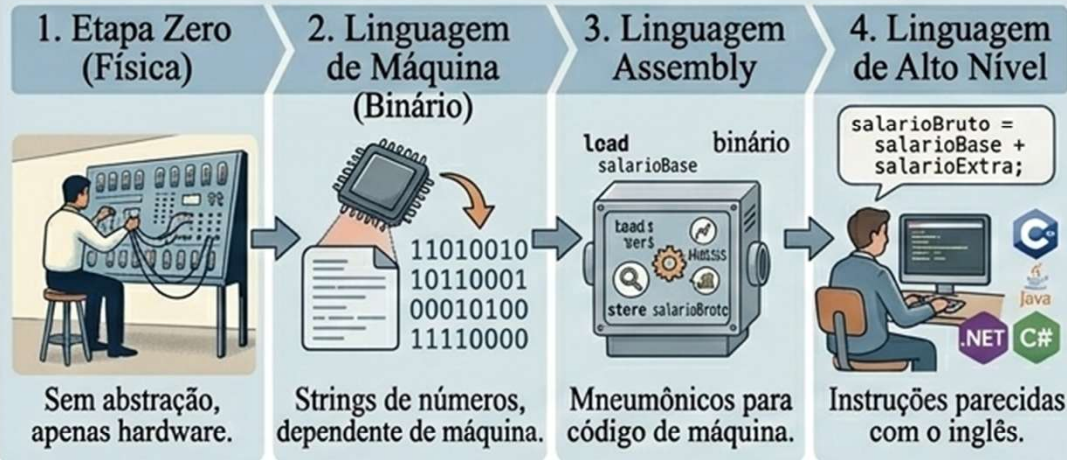
Prof. Dr. Antonio Marcos A. Ferreira

Aula 02: 06/03/2026



O QUE VIMOS ANTERIORMENTE?

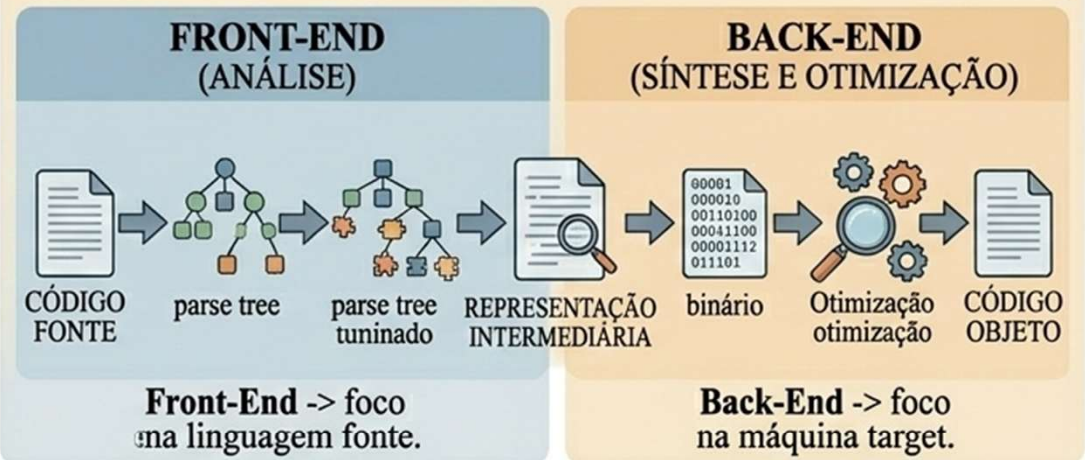
1. CONCEITOS DE LINGUAGENS DE PROGRAMAÇÃO



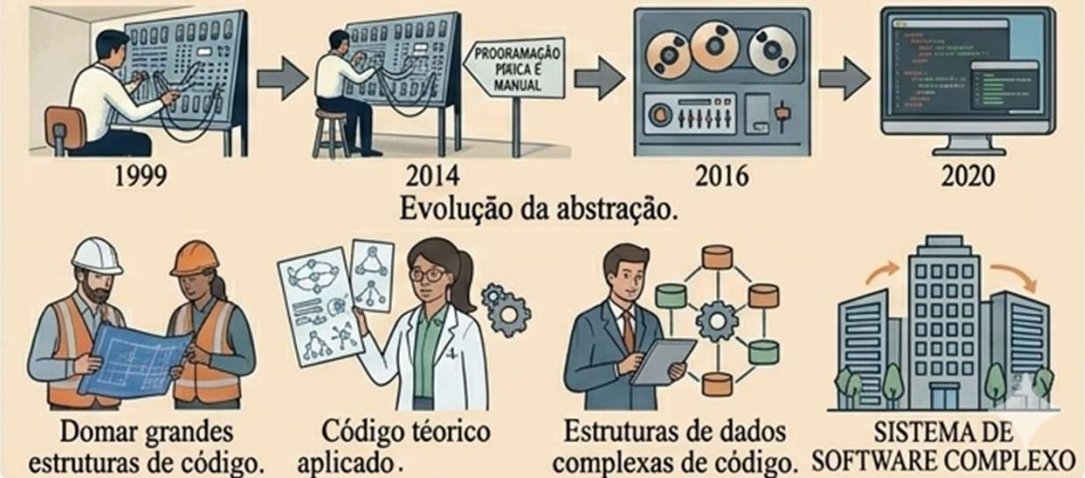
2. FUNÇÃO DOS TRADUTORES (PROCESSADORES DE LINGUAGEM)



3. ESTRUTURA GERAL DE UM COMPILADOR MODERNO (PIPELINE)



4. BREVE HISTÓRICO E MOTIVAÇÃO





Revisão de Linguagens Formais



Alfabetos, símbolos e cadeias

- Alfabeto (Σ): conjunto finito de símbolos (caracteres) permitidos.
 - Ex.: $\Sigma_1 = \{a, b\}$; $\Sigma_2 = \{a \dots z, 0 \dots 9, _, +, -, *, /\}$.
- Símbolo: um elemento de Σ (ex.: a).
- Cadeia (string): sequência finita de símbolos de Σ (ex.: abbaab).
- Cadeia vazia (ϵ): sequência de comprimento zero.
- Σ^* : conjunto de todas as cadeias sobre Σ (inclui ϵ).
- Validação: a1 não é válido sobre $\Sigma = \{a, b\}$; contém símbolo fora de Σ .

Linguagens formais: definidas por conjuntos de cadeias

- Dado um alfabeto Σ , Σ^* é o “universo” de cadeias possíveis.
- Uma linguagem L é qualquer subconjunto de Σ^* que obedece alguma propriedade.
 - Ex.: $L = \{ w \in \{a,b\}^* \mid \text{número de } a \text{ é par} \}$.
- Podemos descrever L por:
 - Expressões regulares (quando for regular),
 - Gramáticas
 - Backus–Naur Form (BNF): É uma notação formal para descrever a sintaxe de linguagens de programação.
 - Extended Backus–Naur Form (EBNF): É uma extensão da BNF para deixar a gramática mais compacta e legível.
 - Autômatos (DFA - Deterministic Finite Automaton /NFA - Nondeterministic Finite Automaton /PDA - Pushdown Automaton), dependendo da classe.

Exemplos de linguagens simples: $\{a^n b^n\}$ vs $\{a^n b^m\}$

- $L_1 = \{ a^n b^n \mid n \geq 1 \}$
 - Mesma quantidade de a seguida da mesma de b.
 - Aceita: ab, aabb, aaabbb. Rejeita: abb, aabbb, ba.
 - Classe: Livre de contexto (não é regular) $\rightarrow$ precisa de pilha (PDA).
- $L_2 = \{ a^n b^m \mid n, m \geq 1 \}$
 - Qualquer n° de a seguido de qualquer n° de b.
 - Aceita: ab, aab, aaabbbb. Rejeita: ba, abab.
 - Classe: Regular $\rightarrow$ reconhecível por AFD.

IMPORTÂNCIA NO DESIGN DE COMPILADORES: DEFINIR A SINTAXE

ESPECIFICAÇÃO FORMAL (TOKENS + GRAMÁTICA)

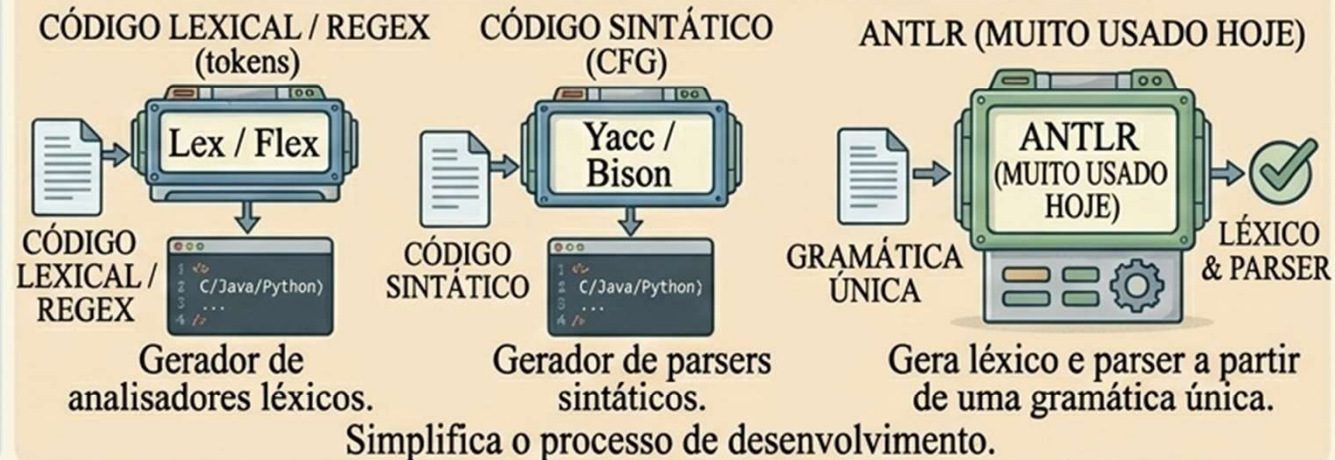
DEFINIR REGRAS E EVITAR AMBIGÜIDADES



Garante uma linguagem documentada e base sólida.

PERMITE GERAR ANALISADORES AUTOMATICAMENTE (FERRAMENTAS)

FERRAMENTAS DE GERAÇÃO (LEX, FLEX, YACC, ANTLR)



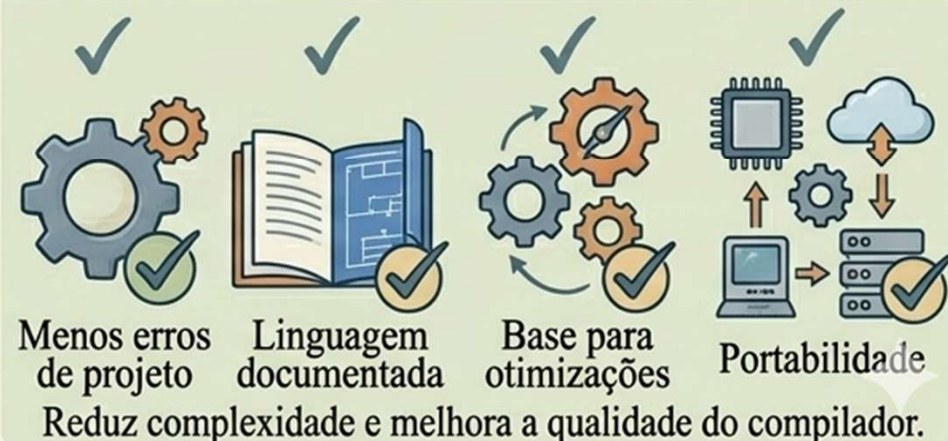
PIPELINE DO COMPILADOR E BENEFÍCIOS PRÁTICOS

FLUXO COMPLETO E RESULTADOS



Tokens/regex -> Léxico (Scanner) -> SCFG/BNF -> Árvore Sintática Abstrata (AST) -> Semântica -> IR/Bytecode (Representação Intermediária).

BENEFÍCIOS PRÁTICOS



Autômato Finito Determinístico (AFD)

Máquina abstrata com número finito de estados.

- Em cada estado, para cada símbolo do alfabeto Σ existe no máximo uma transição.
- Ou seja, não há escolhas: dado o estado atual e o símbolo de entrada, o próximo estado é único.
- Usado diretamente em compiladores, pois garante eficiência na leitura do código fonte.
- Estrutura formal:

$$AFD=(Q, \Sigma, \delta, q_0, F)$$

- Q = conjunto finito de estados
- Σ = alfabeto
- δ = função de transição ($\delta: Q \times \Sigma \rightarrow Q$)
- q_0 = estado inicial
- F = conjunto de estados de aceitação

Exemplo: reconhecer números binários que terminam em 0

- Estados: $\{q_0, q_1\}$
- $\delta(q_0, 0)=q_0, \delta(q_0, 1)=q_1$
- $\delta(q_1, 0)=q_0, \delta(q_1, 1)=q_1$
- $F = \{q_0\}$ (aceita se terminar em 0).

Autômato Finito Não-Determinístico (AFN)

Semelhante ao AFD, mas permite múltiplas transições possíveis para o mesmo símbolo (ou até transições vazias – ϵ -transições).

- Mais flexível para descrever linguagens, mas menos prático para implementação direta.
- Pode ter:
 - Várias escolhas de estados para um mesmo símbolo.
 - Caminhos alternativos possíveis para a mesma cadeia.
- Estrutura formal:

$$AFN = (Q, \Sigma, \delta, q_0, F)$$

- $\delta: Q \times \Sigma \rightarrow P(Q)$ (a transição pode levar a um conjunto de estados, não só 1).
- Exemplo: reconhecer cadeias que terminam em 01
 - Do estado inicial q_0 , lendo 0 pode ir para q_0 ou q_1 (não-determinismo).
 - Se caminho chegar em q_2 com 01, aceita.

RELAÇÃO ENTRE EXPRESSÕES REGULARES E AUTÔMATOS FINITOS



EXPRESSÕES REGULARES (ER)

- São notações formais para descrever conjuntos de cadeias (linguagens regulares).

MUITO USADAS PARA ESPECIFICAR TOKENS EM COMPILADORES (identificadores, números, palavras-chave).

EXEMPLO

Exemplo:

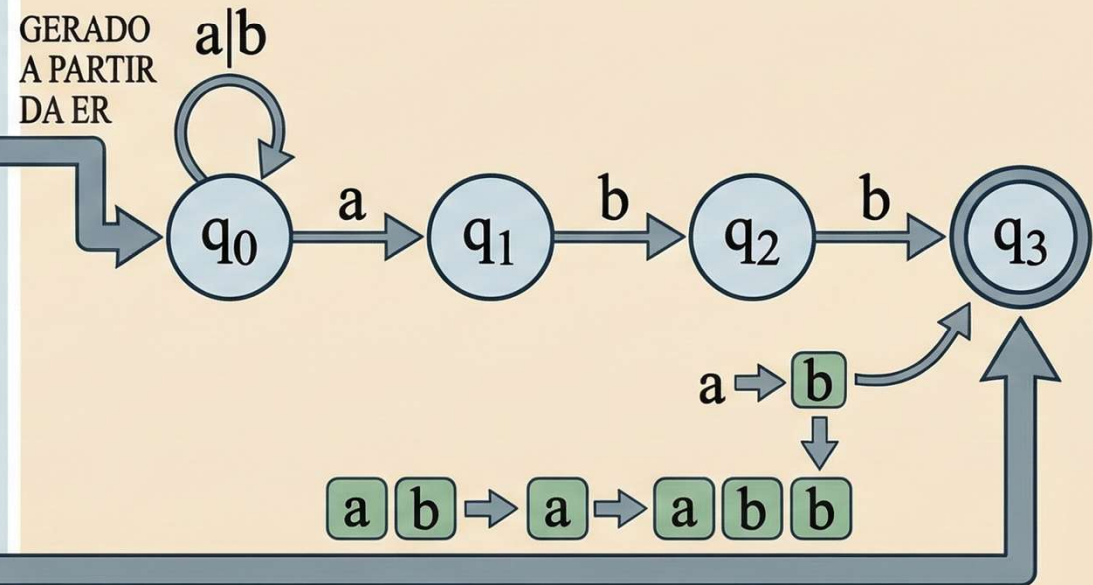
ER: $(a|b)^*abb$

Linguagem: todas as cadeias de a e b que terminam em abb.

✓ abb	✗ ab
✓ aabb	✗ abc
✓ babb	✗ abba
✓ aaabb	✗ a
✓ ababb	✗ b
	✗ abbc

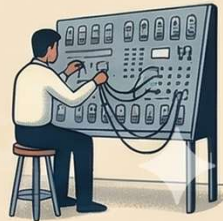
CADEIAS RECONHECIDAS E NÃO RECONHECIDAS PELA ER

AUTOMATOS FINITOS (AF) - O Mecanismo de Reconhecimento



RECONHECE AS CADEIAS VALIDADAS PELA ER

AUTOMATOS FINITOS (AF) SÃO USADOS PELOS PROCESSADORES DE LINGUAGEM PARA RECONHECER CADEIAS ESPECIFICADAS POR EXPRESSÕES REGULARES.



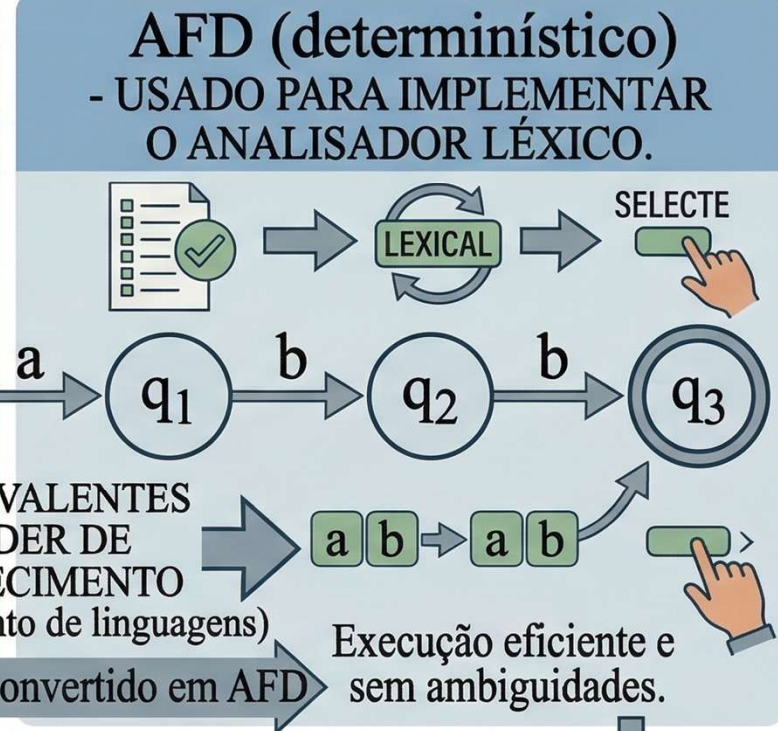
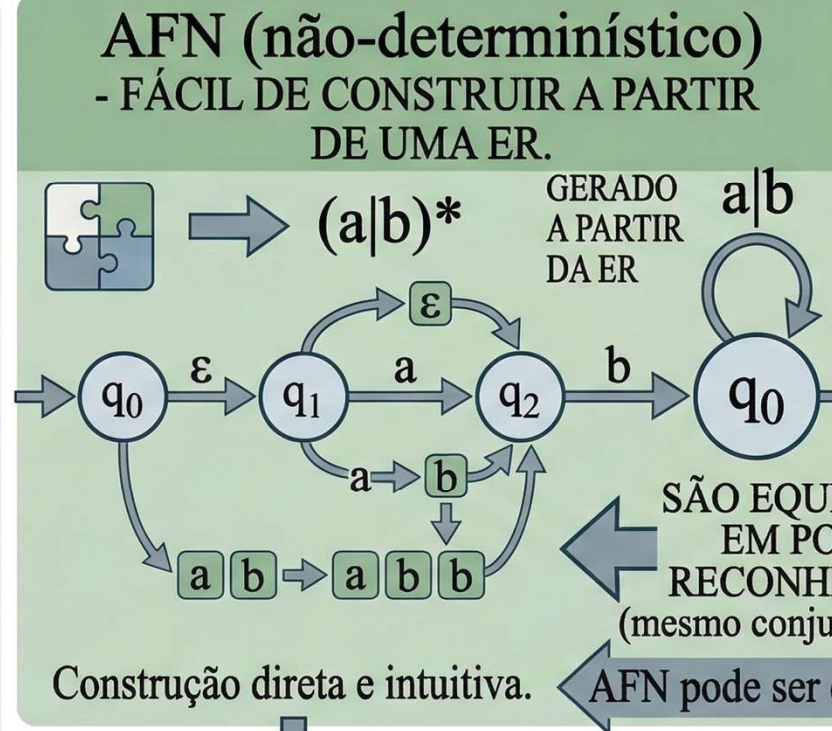
AUTÔMATOS FINITOS (AF) - O Mecanismo de Reconhecimento



AUTÔMATOS FINITOS (AF) - Modelos computacionais que reconhecem linguagens regulares.



São equivalentes em poder de reconhecimento (mesmo conjunto de linguagens).



PAPEL NOS PROCESSADORES DE LINGUAGEM



AFD É USADO NOS PROCESSADORES DE LINGUAGEM PARA RECONHECER CADEIAS ESPECIFICADAS POR EXPRESSÕES REGULARES.

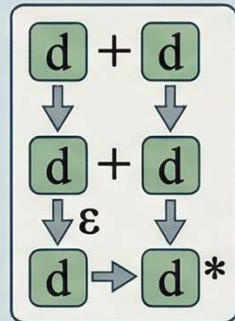
PALAVRA-CHAVE
NÚMERO
IDENTIFICADOR
Token Identificado

RELAÇÃO DIRETA ENTRE EXPRESSÕES REGULARES E AUTÔMATOS FINITOS

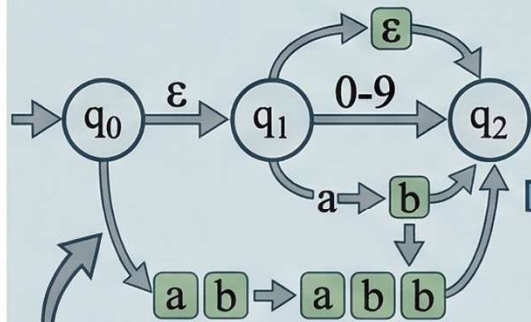
Fase 1: CONVERSÃO DE ER EM AFN (Thompson's Construction)

ER: $[0-9]^+$

construção de Thompson

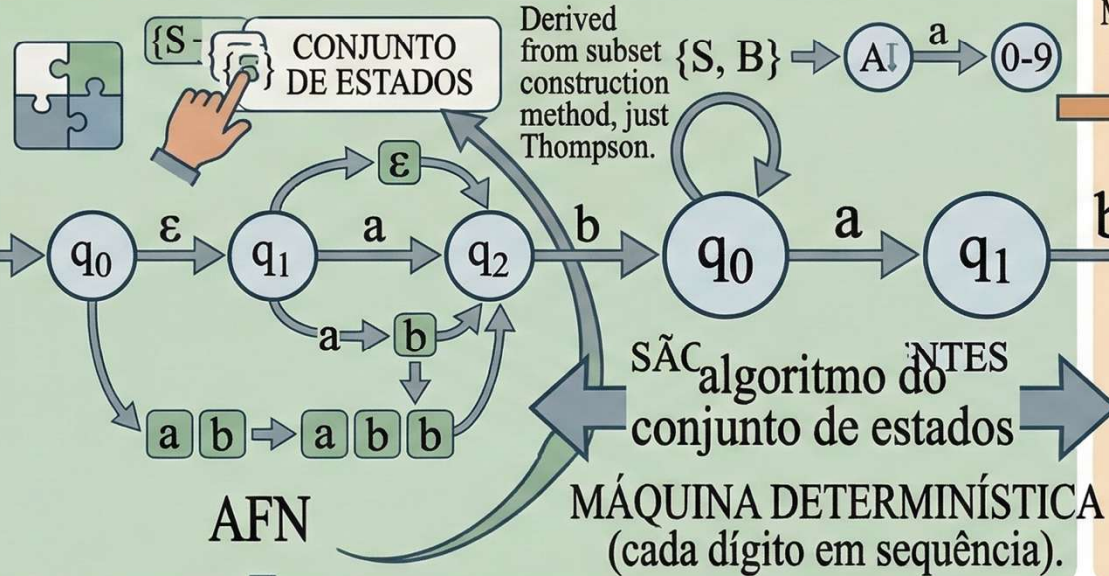


putacionais



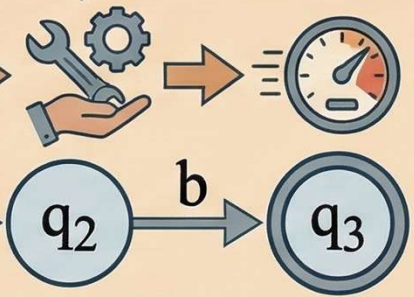
ACEITA UM OU MAIS DÍGITOS (ex: 123, 7).

Fase 2: CONVERSÃO DE AFN EM AFD (Algoritmo do Conjunto de Estados)



Fase 3: AFD FINAL E MINIMIZAÇÃO

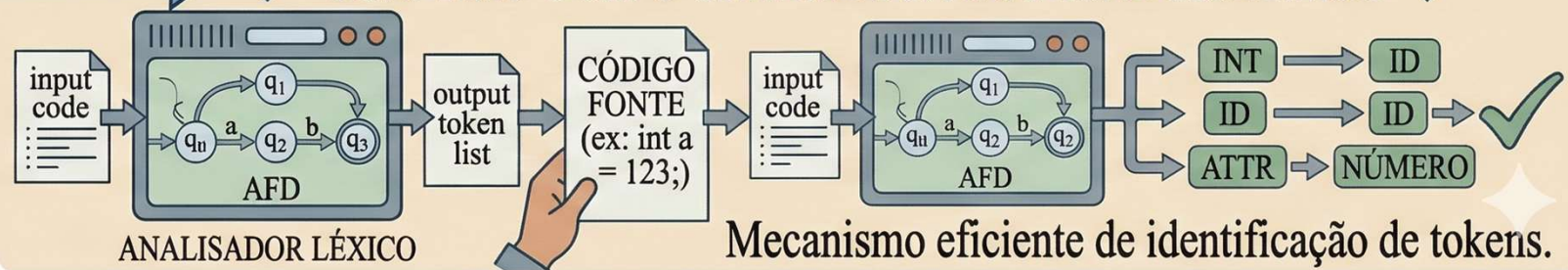
MINIMIZAÇÃO PARA EFICIÊNCIA.



AFD MINIMIZADO

MINIMIZAÇÃO PARA EFICIÊNCIA.

CONEXÃO COM O TRABALHO: ANALISADOR LÉXICO

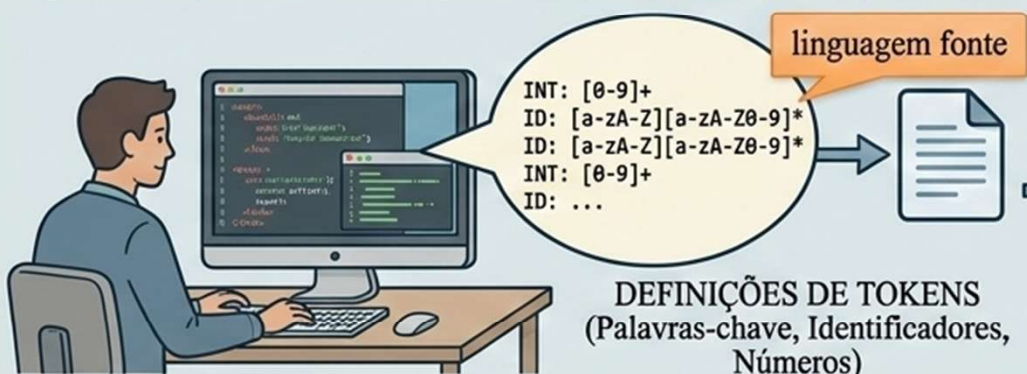


Mecanismo eficiente de identificação de tokens.

RELAÇÃO ENTRE EXPRESSÕES REGULARES E AUTÔMATOS FINITOS NO CONTEXTO DO ANALISADOR LÉXICO

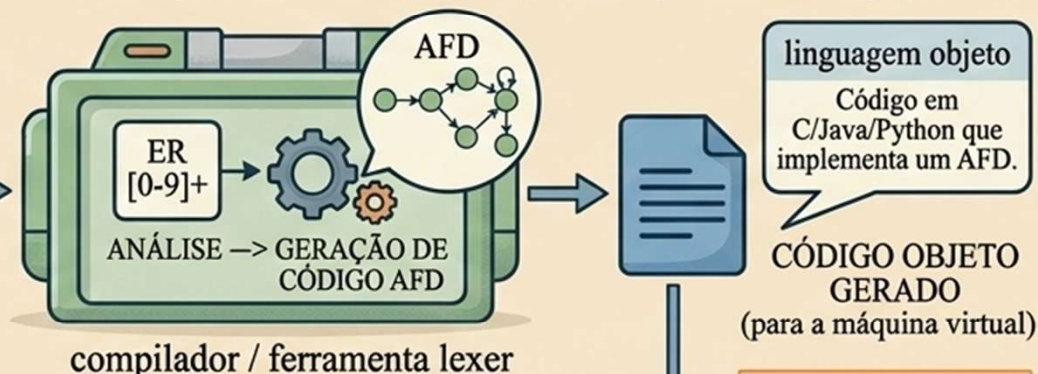
ETAPA 1: O DESENVOLVEDOR ESPECIFICA O LEXER

Expressões Regulares → Entrada para Ferramentas Lexer (ex: Lex, Flex, ANTLR)



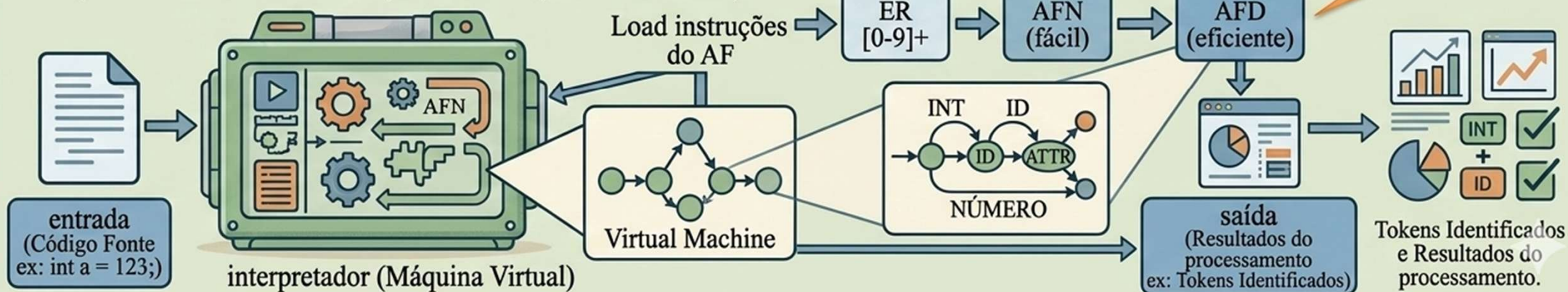
ETAPA 2: TRADUÇÃO PELO COMPILADOR E GERAÇÃO DE CÓDIGO

Compilador Moderno (ex: Flex) → Saída para AFD (Código)



ETAPA 3: EXECUÇÃO DO ANALISADOR LÉXICO PELA MÁQUINA VIRTUAL

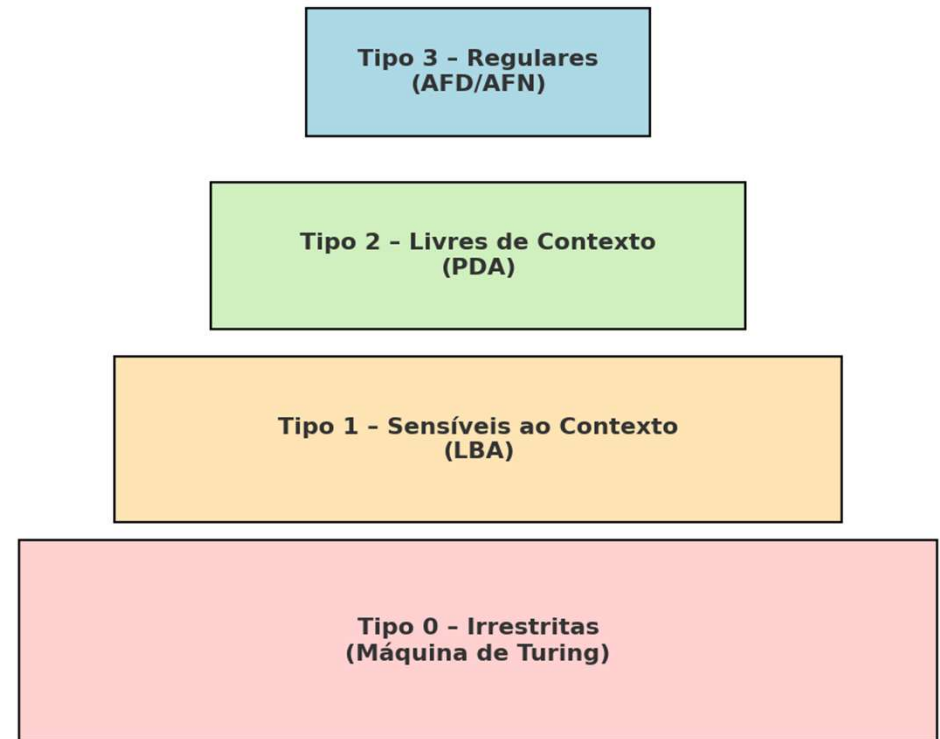
MÁQUINA VIRTUAL → EXECUÇÃO DO AFD (Analisador Léxico)



Hierarquia de *Chomsky*

A Hierarquia de Chomsky (proposta por Noam Chomsky em 1956) é uma classificação das gramáticas formais (e, conseqüentemente, das linguagens que elas descrevem).

- Ela é dividida em quatro tipos principais (do mais geral ao mais restrito)



Tipos de Gramáticas

- Gramáticas Regulares (Tipo 3):
 - Produções simples: $A \rightarrow aB$ ou $A \rightarrow a$.
 - Reconhecidas por autômatos finitos.
 - Aplicação: análise léxica (identificadores, números, palavras-chave).
- Gramáticas Livres de Contexto (Tipo 2):
 - Produções na forma $A \rightarrow \alpha$, onde A é um não-terminal e α é uma sequência de símbolos.
 - Reconhecidas por autômatos de pilha (PDA).
 - Aplicação: análise sintática (estruturas como expressões, if-else, laços).
- Gramáticas Sensíveis ao Contexto (Tipo 1):
 - Produções podem depender do contexto em que aparecem.
 - Reconhecidas por autômatos lineares-limitados (LBA).
 - Menos usadas em compiladores, mas aparecem em análises mais avançadas.
- Gramática Irrestrita (Tipo 0):
 - Não possui restrições nas regras de produção.
 - Reconhecida por uma Máquina de Turing.
 - Abrange qualquer linguagem computável.

APLICAÇÕES EM COMPILADORES: FLUXO DE GRAMÁTICAS

GRAMÁTICAS E RECONHECIMENTO

GRAMÁTICAS REGULARES

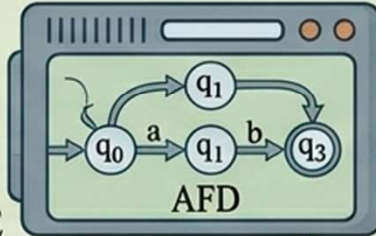
Expressões Regulares
(Tokens: $[0-9]^+$, if)

Autômatos Finitos
(AFN $\rightarrow$ AFD)

Análise Léxica: Foca na identificação de unidades básicas.

RECONHECIMENTO DE TOKENS

CÓDIGO FONTE
(ex: `int a = 123;`)



ANALISADOR LÉXICO

INT
ID
ATTR
NÚMERO

GRAMÁTICAS LIVRES DE CONTEXTO

Linguagens Livre de Contexto
(Estruturas: `if (expr) { stmt }`)

RECONHECIMENTO DE ESTRUTURAS

Análise Sintática: Foca na estrutura e regras da linguagem.

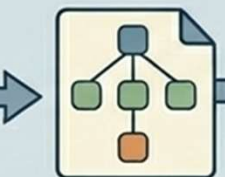
RECONHECIMENTO DE ESTRUTURAS



TOKENS
(ex: INT, ID,
ATTR,
NÚMERO)



ANALISADOR SINTÁTICO
(PARSER)

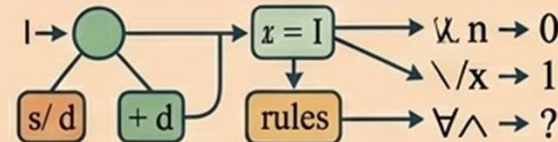


parse tree
(ex: Programa $\rightarrow$
Atribuição $\rightarrow$
[id a, valor 123])

SENSÍVEIS AO CONTEXTO (Menos comuns)

Sensíveis ao Contexto
(ex: type checking,
complex semantics)

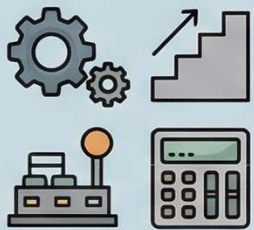
Menos comuns. Usadas para verificações semânticas mais profundas.



PARADIGMAS DE PROGRAMAÇÃO

BREVE REVISÃO DOS PRINCIPAIS PARADIGMAS

IMPERATIVO

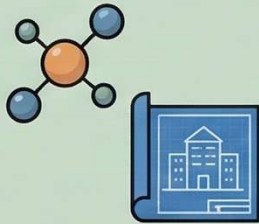


```
int x = 5;  
x = x + 1;
```

(C, Pascal)

Foca em **COMO** fazer as tarefas através de comandos e mudança de estado.

ORIENTADO A OBJETOS

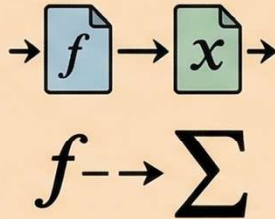


```
class Pessoa {  
  int idade;  
}  
Pessoa p1 = new  
Pessoa();
```

(Java, C++)

Organiza o programa em **OBJETOS** que contêm dados e comportamento.

FUNCIONAL

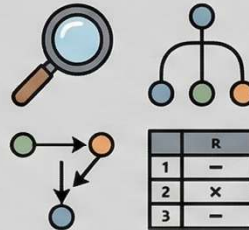


```
add5 x = x + 5  
map add5 [1,2,3]
```

(Haskell, Lisp)

Foca em **O QUE** fazer através de funções e funções matemáticas puras e imutabilidade.

LÓGICO



```
pai(joao, maria).  
pai(joao, jose).  
avo(X, Y) :- pai(X,  
  Z), pai(Z, Y).
```

(Prolog)

Foca em **O QUE** fazer através de fatos e regras regras lógicas de inferência.

CONEXÃO COM O TRABALHO

PROGRAMA EXEMPLO:
CALCULAR FATORIAL

CÓDIGO
IMPERATIVO,
INPUTS &
OUTPUT

CÓDIGO
IMPERATIVO

CÓDIGO
OO

CÓDIGO
FUNCIONAL
CONSTRITRÇÕES
INPUT & OUTPUT

CÓDIGO
FUNCIONAL

CÓDIGO
LÓGICO

CADA PARADIGMA
EXIGE CONSTRUÇÕES
SINTÁTICAS DIFERES.

CÓDIGO
MATEMÁTICAS
E OUTPUTS

**CADA PARADIGMA
EXIGE CONSTRUÇÕES
SINTÁTICAS DIFERENTES.**

READY FOR
COMPILER/INTERPRETER



Conexão com o Projeto de Mini-Compilador



Conexão com o Projeto de Mini-Compilador

- Definir um **alfabeto** e *tokens* iniciais da linguagem escolhida.
- Alfabeto da Linguagem: é o conjunto de símbolos básicos que podem aparecer no código fonte da linguagem. Ele inclui:
 - Letras: a-z, A-Z
 - Dígitos: 0-9
 - Símbolos especiais: + - * / = ; () { }
 - Espaço em branco e quebras de linha(Opcional) outros caracteres, como _, ,, “
- Exemplo de alfabeto para uma mini-linguagem estilo C:

$$\Sigma = \{ a..z, A..Z, 0..9, +, -, *, /, =, ;, (,), \{, \}, <, >, _, \text{ espaço}, \text{ tab}, \text{ quebra de linha} \}$$

Tokens

- É a unidade léxica que o compilador reconhece a partir do alfabeto. Enquanto o alfabeto são os “caracteres básicos”, os tokens são agrupamentos significativos para o compilador.
- Categorias de Tokens
 - 1) Palavras-chave (keywords)
 - if, else, while, return, int, float, etc.
 - 2) Identificadores
 - Nomes definidos pelo usuário: x, y, totalSalario.
 - Regra típica: começam com letra ou _, seguidos de letras/dígitos.

Tokens

3) Literais (constantes)

- Números inteiros: 123
- Números reais: 3.14
- Strings: "hello"

4) Operadores

- Aritméticos: +, -, *, /
- Relacionais: <, >, ==, !=
- Atribuição: =

5) Delimitadores e símbolos especiais

- ;, ,, (,), {, }

6) Espaços em branco e comentários

- Ignorados pelo compilador, mas precisam ser reconhecidos pelo analisador léxico.

Tokens

Exemplo prático:

Suponha o código-fonte:

```
int soma = a + b;
```

- `int` → keyword
- `soma` → identificador
- `=` → operador de atribuição
- `a` → identificador
- `+` → operador aritmético
- `b` → identificador
- `;` → delimitador

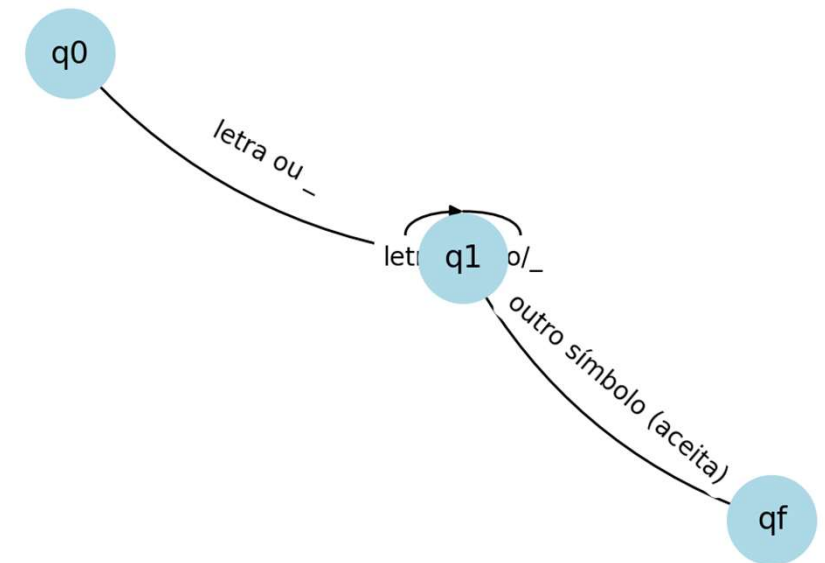
Autômato para Identificadores

- Regra típica:
 - Um identificador deve começar com letra ou `_`.
 - Depois, pode conter letras, dígitos ou `_`.
- Expressão Regular em css:

`[a-zA-Z_][a-zA-Z0-9_]*`

Autômato (AFD simplificado):

- Estado `q0` (inicial):
 - Se receber letra ou `_` → vai para `q1`.
 - Caso contrário → rejeita.
- Estado `q1` (aceitação):
 - Se receber letra/dígito/`_` → permanece em `q1`.
 - Se receber outro símbolo → aceita token e volta para `q0`.



Em palavras:

`q0 --(letra ou "_")--> q1`

`q1 --(letra, dígito, "_")--> q1`

`q1 --(outro símbolo)--> [Aceita identificador]`

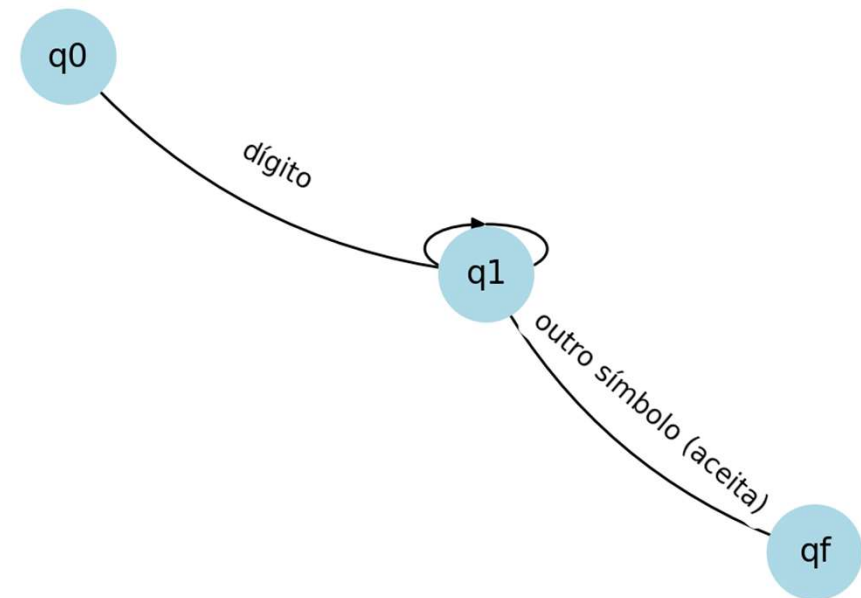
Autômato para Números Inteiros

- Podemos começar simples, só com **inteiros**.
- Expressão Regular em csharp:

[0-9] +

Autômato:

- Estado q0 (inicial):
 - Se receber **dígito** → vai para q1.
 - Caso contrário → rejeita.
- Estado q1 (aceitação):
 - Se receber **dígito** → permanece em q1.
 - Se receber outro símbolo → **aceita número inteiro**.



Em palavras:

q0 --(dígito)--> q1

q1 --(dígito)--> q1

q1 --(outro símbolo)--> [Aceita identificador]

Esboçar a gramática inicial da linguagem

- BNF (*Backus–Naur Form*): define a linguagem usando regras de produção formais.
- EBNF (*Extended BNF*): extensão da BNF, permite notações mais compactas (`[]`, `{}`, `|`).
- Uma gramática é composta por:
 - Símbolos não-terminais (categorias sintáticas, ex.: `<expr>`, `<stmt>`).
 - Símbolos terminais (*tokens* já definidos no léxico: `+`, `id`, `num`, `if`, etc.).
 - Produções que dizem como os não-terminais podem ser substituídos.
 - Símbolo inicial (ex.: `<program>`).

Definindo *tokens* básicos

- Antes da gramática, precisamos dos tokens (já discutidos):
 - Identificadores $\rightarrow$ id
 - Números $\rightarrow$ num
 - Operadores $\rightarrow + - * / = < >$
 - Delimitadores $\rightarrow ; () \{ \}$
 - Palavras-chave $\rightarrow$ if, while, int, float, return

Gramática inicial (BNF simples)

- Programa (PHP-template)

$\langle \text{program} \rangle ::= \langle \text{decl-list} \rangle$

$\langle \text{decl-list} \rangle ::= \langle \text{decl} \rangle \langle \text{decl-list} \rangle \mid \langle \text{decl} \rangle$

- Declarações (kotlin)

$\langle \text{decl} \rangle ::= \langle \text{var-decl} \rangle \mid \langle \text{fun-decl} \rangle$

$\langle \text{var-decl} \rangle ::= \langle \text{type-spec} \rangle \text{ id } ;$

$\langle \text{fun-decl} \rangle ::= \langle \text{type-spec} \rangle \text{ id } (\langle \text{params} \rangle) \{ \langle \text{stmt-list} \rangle \}$

- Tipos (go)

$\langle \text{type-spec} \rangle ::= \text{int} \mid \text{float}$

Exemplo prático

Mostrar um código-fonte real simples (ex.: `int soma = a + b;`) e seguir passo a passo:

- a) Alfabeto →
- b) Tokens →
- c) Autômato que reconhece identificadores e números →
- d) Gramática que descreve a atribuição.

Resposta

a) Alfabeto (Σ)

Conjunto de símbolos básicos aceitos pela linguagem.

$$\Sigma = \{ a..z, A..Z, 0..9, +, -, *, /, =, ;, (,), \{, \}, \text{espaço} \}$$

No código `int soma = a + b`; aparecem:

- Letras (i, n, t, s, o, m, a, b)
- Símbolos especiais (=, +, ;)
- Espaços

Resposta

b) Tokens

Agrupamento de símbolos em unidades significativas:

- `int` → Keyword
- `soma` → Identificador
- `=` → Operador de atribuição
- `a` → Identificador
- `+` → Operador aritmético
- `b` → Identificador
- `;` → Delimitador

Observação! Aqui o analisador léxico transforma caracteres → tokens.

Resposta

c) Autômato (AFD simplificado)

- Identificadores

- Regex: `[a-zA-Z_][a-zA-Z0-9_]*`
- Autômato: começa com letra/_, pode continuar com letras ou dígitos.

- Números inteiros

- Regex: `[0-9]+`
- Autômato: aceita uma sequência de dígitos.

Observação! Esse é o papel do *scanner*: reconhecer padrões de caracteres.

Resposta

d) Gramática (BNF simplificada)

Regras para validar a estrutura da atribuição:

```
php-template
<stmt> ::= <decl> | <assign> ;
<decl> ::= int <id> ;
<assign> ::= <id> = <expr> ;
<expr> ::= <id> + <id> | <id>
<id> ::= [a-zA-Z_][a-zA-Z0-9_]*
```

- Aplicando ao código `int soma = a + b;`
- Reconhece primeiro `int soma;` como declaração.
- Depois `soma = a + b;` como atribuição com expressão aritmética.